



Frontend Engineering Take-Home Exercise

Shipment Management UI

Purpose

This exercise evaluates your ability to build a React application that handles list/detail layouts, state management, data filtering, and business rule enforcement. We are looking at your component design, state management decisions, and UI engineering judgment - not just whether the features work.

Duration: **7 calendar days** from when you receive this exercise. This is wall-clock time, not expected effort - we know you may have a full-time job. We estimate the core requirements take 4-8 hours of focused work.

Business Context

Jitsu coordinates last-mile package delivery. Shipments arrive at a warehouse, get assigned to delivery routes (called assignments), and are delivered by drivers.

Operations staff use internal tools to manage shipments and assignments. This exercise asks you to build a simplified version of one of those tools.

Domain

Shipment

A package to be delivered. Each shipment has a status, a client, a warehouse, location coordinates, and dates.

OPEN	Shipment received at warehouse; not yet assigned to a route
IN_TRANSIT	Assigned to a driver; delivery in progress
DELIVERED	Successfully delivered to the recipient

A shipment's status follows this lifecycle:



- OPEN → IN_TRANSIT: the shipment is assigned to a driver via an assignment. When transitioning to IN_TRANSIT, an `assignment_id` must be set.
- IN_TRANSIT → DELIVERED: the driver has completed the delivery.
- IN_TRANSIT → OPEN: the shipment is unassigned (reverted). The `assignment_id` must be cleared.

Other transitions (e.g. OPEN → DELIVERED, DELIVERED → OPEN) are not valid.

Assignment

A delivery route grouping one or more shipments. Assignments also have a status (OPEN or COMPLETED) and a list of associated clients.

Shipment Fields

```
id string Unique identifier (e.g. "shp_003")
client_name string Client who owns the shipment
label string Human-readable label (e.g. "LAX-581-250521-6") status
string OPEN | IN_TRANSIT | DELIVERED
arrival_date datetime When the shipment arrived at the warehouse
delivery_by_date datetime Delivery deadline
eta datetime Estimated delivery time
warehouse_id string Warehouse identifier
assignment_id string? Assignment this shipment belongs to (null if OPEN)
lat number Delivery latitude
lng number Delivery longitude
```

Assignment Fields

```
id string Unique identifier (e.g. "as_002")
label string Human-readable label (e.g. "TX-127")
status string OPEN | COMPLETED
clients string[] Client names associated with this assignment
shipment_count number Number of shipments in this assignment
```

Requirements

The exercise is structured in three tiers. Complete the Core requirements, then continue to Stretch and Extra Credit as time allows.

Core - expected from all candidates

Stretch - expected from senior candidates; encouraged for mid-level

Extra Credit - demonstrates advanced judgment; not required at any level

© 2026 Jitsu. All rights reserved. Page 2



Core: Shipment List Page

Build a shipment management page with two panels:

Left Panel - Shipment List

- Display shipments grouped by status (OPEN, IN_TRANSIT, DELIVERED)
- Each shipment row shows: client name, label, and arrival date
- Shipments are clickable to view details in the right panel
- Include a search input that filters shipments by label or client name
- Handle a huge number of shipments (sometimes we have over 100k shipments per day).

Right Panel - Shipment Detail

When a shipment is selected, display:

- All shipment fields: `client_name`, `label`, `status`, `arrival_date`, `delivery_by_date`, `warehouse_id`, `assignment_id`
- Editable fields: `delivery_by_date`, `lat`, `lng`
- A save action that persists changes via API call

Stretch: Status Transitions and Map

Status Updates

- Add a status dropdown to the shipment detail panel
- Enforce valid transitions: OPEN → IN_TRANSIT (requires selecting an assignment), IN_TRANSIT → DELIVERED, IN_TRANSIT → OPEN (clears assignment)
- Prevent invalid transitions (e.g. OPEN → DELIVERED)
- Display only valid target statuses in the dropdown

Map

- In the shipment detail panel, display a map showing the selected shipment's location pin • You may use any map library (Leaflet, Mapbox, Google Maps, etc.)

CRUD

- Allow inserting new shipments (with sensible defaults)
- Allow deleting shipments

© 2026 Jitsu. All rights reserved. Page 3



Extra Credit: Assignment Page

Add a second page (with routing) for managing assignments:

- **Panel 1 - Assignment List:** display assignments grouped by status, searchable by label •
 - **Panel 2 - Assignment Detail:** show assignment details and list of shipments in the assignment
 - **Panel 3 - Shipment Detail:** when clicking a shipment within an assignment, show its detail.
The map in this view should show all shipments in the assignment connected by lines, centered on the selected shipment.
- Allow creating new assignments
 - Allow deleting empty assignments

Sample Data

A data generation script is provided below. You can use the generated JSON file as mock data, or serve it via [json-server](#) for a realistic API experience.

generated-data.js

```

const fs = require("fs");
const statusList = ["OPEN", "IN_TRANSIT", "DELIVERED"];
const statuses = statusList.map((status) => ({ id: status }));
const clients = [
  "Sony", "Samsung", "DHL", "CargoTrans", "ShipCo", "Logix",
  "Oceanic", ];
const warehouses = ["EWR", "LAX", "JFK", "SFO", "SEA"];
const baseDate = new Date();
const minLat = 32.55, maxLat = 33.05;
const minLng = -97.40, maxLng = -96.50;
const shipments = [];

for (let i = 1; i <= 100; i++) {
  const arrival = new Date(baseDate);
  arrival.setDate(arrival.getDate() - Math.floor(Math.random() *
    10)); const eta = new Date(arrival);
  eta.setHours(eta.getHours() + Math.floor(Math.random() * 48));
  shipments.push({
    id: `shp_${String(i).padStart(3, "0")}`,
    client_name: clients[i % clients.length],
    label: `${warehouses[i % warehouses.length]}-581-2505${20 + (i %
    10)}-${i}`, status: statusList[i % statusList.length],
    arrival_date: arrival.toISOString(),

```

© 2026 Jitsu. All rights reserved. Page 4



```

    delivery_by_date: new Date(arrival.getTime() + 2 *
      86400000).toISOString(), eta: eta.toISOString(),
    warehouse_id: "581",
    lat: Math.random() * (maxLat - minLat) + minLat,
    lng: Math.random() * (maxLng - minLng) + minLng,
  });
}

const result = { statuses, shipments };
fs.writeFileSync("shipments.json", JSON.stringify(result, null,
  2)); console.log("shipment data generated");

```

Serving via json-server

```

npm install -g json-server
json-server --watch shipments.json --port 3001

# Available endpoints:
# GET /shipments?_page=1&_per_page=25
# GET /shipments/:id
# PUT /shipments/:id
# POST /shipments
# DELETE /shipments/:id
# GET /statuses

```

Deliverable

Push your project to a public GitHub repository. Your submission should include:

- **Working React application** implementing at minimum the Core requirements
- **README.md** that explains: prerequisites, how to install dependencies, how to run the application, and a brief description of your approach and any tradeoffs you made
- **A short video recording** (2-5 minutes) demonstrating the application. Walk through the features you implemented and briefly explain one or two design decisions.

If something is ambiguous, make a reasonable assumption, document it in your README, and proceed.

How We Review Submissions

A well-designed Core implementation is better than a rushed attempt at everything. We value engineering judgment over feature completeness.

© 2026 Jitsu. All rights reserved. Page 5



We look at how you:

- Structure your components and manage state
- Handle the data flow between list, detail, and edit views
- Enforce business rules (status transitions, assignment requirements)
- Make the UI usable and responsive to user actions
- Organize your code for readability and maintainability
- Document your decisions

Assumptions You May Make

- Use the provided data generator or serve via json-server. You do not need to build a real backend.
- You may use any React-compatible libraries (routing, state management, UI components, map libraries). Your choices are part of what we evaluate.
- TypeScript is preferred but not required.
- Visual polish is appreciated but not the primary evaluation criterion. A functional, well-structured application with basic styling is fine.

- You do not need to implement authentication or user management.

Notes for Candidates

We fully expect that you may use LLMs or other tools to help with this exercise. That is fine. However, you should be able to explain every design decision in your code during the follow-up conversation. The video walkthrough and live discussion are where we evaluate your understanding.

A clear, well-reasoned solution for the Core tier is better than an exhaustive solution that you cannot explain.

You do not need prior experience with Jitsu or the logistics industry to do well on this exercise. © 2026